

Examen parcial - Operación de Modelos

MUIAAP · ICAI / Comillas · Viernes, 9 de octubre de 2026

Nombre y apellidos: _____

Puntuación: 10 puntos. Teoría S1: 3 puntos. Práctica S2-S3: 7 puntos.

Duración total: 120 minutos. La teoría dispone de un máximo de 30 minutos; el tiempo restante se dedica a la práctica (al menos 90 minutos). Las condiciones sobre consulta de material, conexión a Internet y asistentes de IA las comunicará el profesor antes del examen.

Parte A. Conceptos básicos de MLOps y MLflow (3 puntos)

Seis preguntas de 0,5 puntos. Tiempo máximo: 30 minutos. No se pide código. Formatos: verdadero/falso, completar hueco, tipo test y emparejar.

- En A1 escribe Verdadero o Falso. No se pide explicación adicional.
- En A2, A3 y A6 indica una sola opción (a, b, c o d).
- En A4 y A5 escribe las correspondencias de cada elemento.
- Escribe todas las respuestas A1-A6 en RESPUESTAS_TEORIA.md.

A1. Del prototipo a producción - verdadero/falso (0,5 puntos)

Indica si la siguiente afirmación es verdadera o falsa:

"Un buen resultado en el notebook es suficiente para llevar el modelo a producción sin más comprobaciones."

A2. Ciclo de vida del modelo - completar hueco (0,5 puntos)

¿Qué etapa falta en este ciclo de vida?

Preparar datos y dividir → entrenar → comparar con validación → ____ → registrar → desplegar y monitorizar.

- a) Entrenar de nuevo con test.
- b) Evaluación final en test del modelo elegido.
- c) Borrar los datos de validación.
- d) Cambiar el umbral.

A3. Uso del test - tipo test (0,5 puntos)

¿Para qué se usa el conjunto de test?

- a) Para elegir el mejor modelo entre varios.
- b) Para ajustar hiperparámetros.
- c) Para medir al final el rendimiento del modelo ya elegido.
- d) Para entrenar con más datos.

A4. Experimento y run en MLflow - emparejar (0,5 puntos)

Empareja cada término con su definición: Experimento / Run.

- a) Una ejecución concreta con su configuración y sus resultados.
- b) Un grupo de runs comparables entre sí.

A5. Parámetros, métricas y artefactos - emparejar (0,5 puntos)

Empareja cada elemento con su tipo:

- max_depth=4
- F1_validación=0.82
- matriz_confusion.png

Tipos: parámetro / métrica / artefacto.

A6. Criterio de selección - tipo test (0,5 puntos)

El criterio es $F1_validación \geq 0,80$. Candidatos: A = 0,79, B = 0,80, C = 0,78. ¿Qué se hace?

- a) Se elige A, el que más se acerca.
- b) Se elige B, el único que cumple.
- c) Se baja el umbral a 0,78 para aprobar los tres.
- d) Se despliega C.

Parte B. Práctica - semanas 2 y 3 (7 puntos)

Caso: clasificación de paquetes

Una empresa de reparto recibe un CSV de paquetes y quiere clasificarlos como normal o urgente usando un modelo ya entrenado. Debes construir un pequeño proyecto local que valide los datos, haga inferencia y guarde las predicciones. No tienes que entrenar ningún modelo ni programar una API, una interfaz web, MLflow o un manifiesto.

Recibes archivos sueltos: `contracts.py`, `inference.py`, `test_inference.py`, `paquetes.csv`, `paquetes_invalidos.csv` y `modelo.joblib`. El modelo es didáctico, no una regla operativa real. Su entrenamiento y sus etiquetas no forman parte del examen.

B1. Proyecto reproducible (1 punto)

Crea un proyecto con Python y uv. Organiza los archivos con esta estructura final; no cambies los nombres públicos del starter:

```
clasificador-paquetes/  
  README.md  
  pyproject.toml  
  uv.lock  
  .gitignore  
  src/clasificador_paquetes/__init__.py  
  src/clasificador_paquetes/contracts.py  
  src/clasificador_paquetes/inference.py  
  tests/test_inference.py  
  data/raw/paquetes.csv  
  data/raw/paquetes_invalidos.csv  
  models/modelo.joblib
```

Deja un README con los comandos para instalar, ejecutar y probar desde la raíz del proyecto. Inicializa Git y registra al menos dos commits coherentes. No incluyas `.venv/`, cachés ni credenciales. Publica el proyecto en GitHub para la entrega. No se pide abrir un PR.

Crea el proyecto con `uv init --package clasificador-paquetes`.

Instala las dependencias de `FICHA_ENTORNO.md` y genera el lockfile. Usa la versión de Python que tengas instalada.

B2. Contratos Pydantic (1,5 puntos)

Completa las clases de `contracts.py` con Pydantic v2. Los números llegan como texto desde el CSV: deben aceptarse textos numéricos y rechazarse textos no numéricos. En ambas se rechazan campos extra.

Entrada

- `id_paquete`: texto. Elimina los espacios al principio y al final; después no puede estar vacío.
- `peso_kg`: número finito, mayor que 0 y menor o igual que 30.
- `distancia_km`: número finito, entre 0 y 200, ambos incluidos.

Salida

- `id_paquete`: texto no vacío.
- `categoria`: solo normal o urgente.
- `confianza`: número finito entre 0 y 1, ambos incluidos.

B3. Lectura y validación (1 punto)

Completa `leer_csv(ruta)`. Devuelve una lista de objetos `Entrada`, en el mismo orden que las filas del archivo. Cada fila debe pasar por el contrato de entrada. Si alguna fila es inválida, la función debe fallar: no la ignores ni la corrijas inventando valores. Un CSV sin filas de datos debe producir un `ValueError`. Puedes usar `csv.DictReader`.

B4. Preprocesado (0,75 puntos)

Completa `preprocesar(entrada)`. Recibe una `Entrada` ya validada y devuelve una lista de dos números: primero `peso_kg` redondeado a una cifra decimal (tip: `round`) y después `distancia_km`, sin modificar la distancia. No incluyas el identificador en el vector. No cambies el objeto recibido.

Ejemplo: peso 2.36 y distancia 8 producen `[2.4, 8.0]`.

B5. Modelo e inferencia (1,25 puntos)

Completa `cargar_modelo(ruta)` y `predecir(entrada, modelo)`.

- Carga con `joblib` el modelo proporcionado
- El modelo recibe una matriz (2D): una lista que contiene el vector de dos características, en el orden de B4.
- `modelo.predict(matriz)` devuelve una categoría por fila. `modelo.predict_proba(matriz)` devuelve las probabilidades en el mismo orden que `modelo.classes_`.
- Para una única entrada, devuelve un objeto `Salida` con su identificador, la categoría predicha y, como confianza, la mayor probabilidad de esa fila. La salida debe validarse con `Pydantic`; no devuelvas un diccionario sin validar.

B6. Ejecución y salida CSV (1 punto)

Completa `guardar_csv(resultados, ruta)` y `ejecutar(entrada, modelo, salida)`.

- El CSV de salida tendrá cabecera `id_paquete`, `categoria`, `confianza` y una fila por paquete, respetando el orden original.
- `ejecutar` debe leer y validar todas las entradas, cargar el modelo una sola vez, predecir cada paquete y guardar al final.
- Si falla una entrada o una predicción, no debe crearse un archivo de resultados parcial. Si la ruta de salida ya existía, su contenido debe permanecer intacto ante ese fallo. No se pide proteger frente a un fallo del disco mientras se escribe.
- La ejecución desde la raíz será `uv run python -m clasificador_paquetes.inference`. El bloque de arranque y las rutas ya están en el starter; no se pide crear argumentos de terminal.

B7. Pruebas (0,5 puntos)

Se entrega `test_inference.py` con las pruebas completas. Colócalo en `tests/` y, desde la raíz del proyecto, ejecuta:

```
uv run pytest
```

Las pruebas deben pasar cuando hayas completado el código. Si aparece `FAILED`, revisa la prueba indicada y el error para corregir tu código. No borres ni cambies las pruebas para ocultar fallos. Ejecutar el programa no sustituye esta comprobación.

Comprobación y entrega

Con el CSV válido debes obtener seis filas de salida, identificadores sin espacios y solo las categorías permitidas. Con `paquetes_invalidos.csv` la ejecución debe fallar por la segunda fila, sin salida parcial. No se evalúa entrenar un modelo mejor.

Sube el proyecto a tu repositorio de GitHub. Incluye el modelo suministrado, el README, el lockfile, el código y las pruebas, sin `.venv/`, cachés ni credenciales. Escribe las respuestas A1-A6 en `RESPUESTAS_TEORIA.md` y súbelo al mismo repositorio para corregir teoría y práctica desde allí. Entrega al profesor el enlace del repositorio; el canal lo indicará el profesor.

La corrección se hace por apartado: un fallo en el flujo completo no anula automáticamente los apartados que funcionan. Se valoran resultados y contratos, no una única forma de escribir el código. Puedes usar bucles y condiciones sencillos.